

[← Go Back](#)

Hardware

Portenta Machine Control ▼

Tutorials

Memory Partitioning for Use with the Arduino IDE

Arduino Portenta Machine Control Library Guide

Modbus RTU On Portenta Machine Control Using PLC IDE

Modbus TCP On Portenta Machine Control Using PLC IDE

Modbus TCP with Portenta Machine Control & Opta™

Tank Thermoregulation with Portenta Machine Control & Opta™

Connect an RTD/Thermocouple to the Portenta Machine Control

Portenta Machine Control User Manual

Home / Hardware / Portenta Machine Control / **Memory Partitioning for Use with the Arduino IDE**

Memory Partitioning for Use with the Arduino IDE

Learn how to partition the memory of an Opta™ or a Portenta Machine Control to be used with the Arduino® ecosystem tools.

Author José Bagur and Taddy Chung · Last revision · 05/30/2025

ON THIS PAGE

Overview

Goals

Hardware and Software Requirements +

Memory Partitioning

Setting Up the Arduino IDE

Partitioning the Memory of an Opta™

Partitioning the Memory of a Portenta Machine Control

Conclusion

Next Steps

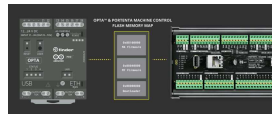
Overview

The **Opta™** is a secure, easy-to-use micro Programmable Logic Controller (PLC) with Industrial Internet of Things (IIoT) capabilities. The **Portenta Machine Control** is a centralized, energy-efficient industrial control unit capable of operating equipment and machinery.

Both devices can be programmed with the **Arduino PLC IDE**, a tool that simplifies programming the device through any of the five programming languages defined by the IEC 61131-3 standard.

[Help](#)

programmed with other tools from the Arduino ecosystem, such as the [Arduino IDE](#) and the Arduino programming language. Suppose we are using Opta™ with the Arduino PLC IDE and want to start programming the device using the Arduino IDE and the rest of the Arduino ecosystem tools. In that case, we need to partition the device's memory to enable all available features. This applies to the Portenta Machine Control as well.



The Memory Map for
Opta™ & Portenta
Machine Control

In this tutorial, we will use an Opta™ and Portenta Machine Control to learn how to partition the memory, enabling all functionalities to be programmed using the Arduino IDE and the Arduino programming language.

Goals

Learn how to partition the memory of an Opta™ and a Portenta Machine Control for use with the Arduino IDE and other ecosystem tools.

Program an Opta™ and a Portenta Machine Control using the Arduino IDE and the Arduino programming

Hardware and Software Requirements

Hardware Requirements

Opta™ ([Opta™ Lite](#), [Opta™ RS485](#), or [Opta™ WiFi](#)) (x1)
or [Portenta Machine Control](#) (x1)

USB-C® cable (x1)

[Micro USB cable](#) (x1)

Software Requirements

Before beginning the memory partitioning process, ensure you have the following software requirements:

[Arduino IDE 1.8.10+](#),
[Arduino IDE 2](#), or [Arduino Cloud Editor](#)

Arduino Mbed OS Opta Boards

core

Arduino Mbed OS Portenta Boards

core

The **QSPIFormat** sketch required for memory partitioning is available in core **version 4.3.1 or higher**. To install or



update your core, open the Arduino IDE, go to **Tools > Board > Boards Manager**, search for

[Opta](#) or [Portenta](#),

and install or update to

Memory Partitioning

Memory partitioning involves dividing a device's available memory into separate regions or partitions, each serving a specific purpose. This process is used to manage memory efficiently and ensure that different programs or functionalities of the device can coexist without interfering.

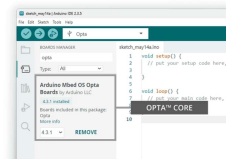
In the context of the Opta™, partitioning the memory allows the device to accommodate different functionalities, such as its Wi-Fi® module firmware, Over-The-Air (OTA) updates functionality, and certificate storage, while still providing memory space for user programs developed in the Arduino IDE and the Arduino programming language. Similarly, it applies to the Portenta Machine Control as well.

Now, we will guide you through partitioning the memory of an Opta™ and Portenta Machine Control to enable full programming capabilities using the Arduino IDE and the Arduino programming language.

Setting Up the Arduino IDE

This tutorial requires the latest version of the Arduino IDE, which can be downloaded [here](#).

In the Arduino IDE, you need to install the core for *Opta™*. To do this, go to **Tools > Board > Boards Manager** or click the **Boards Manager** icon in the left tab of the IDE. In the **Boards Manager** tab, search for `opta` and install the latest `Arduino Mbed OS Opta Boards` version.



Installing the Opta™ core in the Arduino IDE bootloader

For the *Portenta Machine Control*, in the **Boards Manager** tab, search for `portenta` and install the latest

`Arduino Mbed OS Portenta Boards`

core version.

We are now ready to compile and upload Arduino sketches to an Opta™ or a Portenta Machine Control using the Arduino IDE.

Memory partitioning of an Opta™ or a Portenta Machine Control should use the **latest core** and **libraries** to ensure the system is up-to-date with the intended default configuration.



Partitioning the Memory of an Opta™

To partition the memory of an Opta™ device, you need an Arduino sketch that includes memory partition operations and a certificate file. The required sketch is called **QSPIFormat** and can be found in the Arduino IDE by navigating to **File > Examples > STM32H747_System**. The necessary files are organized in the following directory structure:

```
1 QSPIFormat
2 |— certificates.h
3 |— QSPIFormat.ino
```

If you cannot find the **QSPIFormat** sketch in your board's examples, please make sure you have installed the **latest Arduino Mbed OS Opta Boards core (version 4.3.1 or higher)**.

The sketch below shows the memory partition process to be applied for Opta™.

```

1  #include "BlockDev
2  #include "MBRBlockL
3  #include "LittleFil
4  #include "FATFileSy
5  #include "wiced_res
6  #include "certifica
7
8  #ifndef CORE_CM7
9      #error Format QSP
10 #endif
11
12 using namespace mbe
13
14 BlockDevice* root =
15 MBRBlockDevice wifi
16 MBRBlockDevice ota_
17 MBRBlockDevice kvst
18 MBRBlockDevice user
19 FATFileSystem wifi_
20 FATFileSystem ota_d
21 FileSystem * user_d
22
23 bool waitResponse()
24     bool confirmation
25     bool proceed = fa
26     while (confirmati
27         if (Serial.avai
28         char choice =

```

The sketch shown above performs the following tasks:

1 Initialize and erase the QSPI Flash

memory: The sketch initializes the QSPI Flash memory of the Opta™ and erases its content to prepare the memory for new firmware and data. One of the built-in LEDs of the device is used to indicate the progress of the memory-erasing process.

2 **Create partitions and format them in the QSPI Flash**

memory: The sketch creates four partitions in the QSPI Flash memory:

Partition 1: Wi-Fi firmware and certificates (1MB)

Partition 2: OTA (Over-The-Air) (5MB)

Partition 3: Provisioning KVSStore (1MB)

Partition 4: User data / OPTA PLC runtime (7MB)

1 **Write Wi-Fi firmware and certificate data:**

The sketch writes the Wi-Fi firmware and certificate data to the appropriate partitions in the QSPI Flash memory and flashes the memory-mapped Wi-Fi firmware and certificates.

2 **Format the user data**

partition: The sketch prompts you to choose between **LittleFS** or **FatFS** for formatting the user data partition. When using the Opta™ with the PLC IDE, you should select ☐ **(No)** when prompted to choose user data partition format. This will use **FatFS** instead, which is required for

As indicated in the sketch

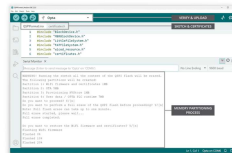


output, **LittleFS** is not supported by the OPTA PLC runtime.

1 Display progress in the Arduino IDE

Serial Monitor: The sketch provides a visual indication of the progress of the flashing process using one of the built-in LEDs of the Opta™ and through messages in the Arduino IDE Serial Monitor.

To upload the code, click the **Verify** button to compile the sketch and check for errors. Then, click the **Upload** button to program the device with the sketch. If two COM ports are available, which might happen after using the PLC IDE to program the device, select the COM port with the lower number.



Uploading a sketch to the Opta™ the Arduino IDE

When you run the **QSPIFormat** sketch on your Opta™ device, follow the prompts displayed in the Arduino IDE's Serial Monitor:

content of the QSPI flash will be erased, as well as information about the partitions that will be created.

- 2 The Serial Monitor will ask,

```
Do you want to  
proceed? Y/[n]
```

, type **Y** and press *Enter* to continue.

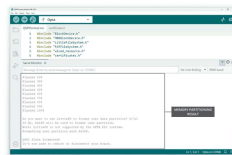
- 3 Next, it will ask,

```
Do you want to  
perform a full  
erase of the QSPI  
flash before  
proceeding? Y/[n]
```

. Please answer according to your needs.

- 4 The system will then erase the flash memory and show progress indicators for the Wi-Fi firmware, certificates and memory mapped Wi-Fi firmware.

At the end of the memory partitioning process, you'll be prompted if you want to use **LittleFS** to format user data partition.

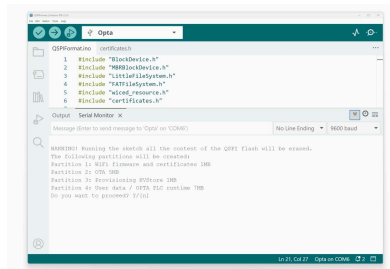


Memory partitioning
results of the Opta™
as shown in the
Arduino IDE's Serial
Monitor

When planning to use the Opta™
with the PLC IDE, you must enter

required for proper compatibility with the PLC runtime. If you need to use **LittleFS** to format the user data partition, please enter ☐ Y.

You should see a brief success message in the Serial Monitor if everything went as intended. Your Opta™ device should now be properly partitioned and formatted, making it ready for use. The process typically takes a few minutes to complete, with progress indicators keeping you informed. The following clip shows the complete memory partitioning process from start to finish:



Note: If you encounter issues with the Opta™ when using the PLC IDE after memory partitioning with the **QSPIFormat**

i sketch, please use the dedicated Opta memory partitioning code available for download [here](#). Please store both files in the same directory.

Partitioning the Memory of a Portenta Machine

The memory partitioning process for the Portenta Machine Control follows the same procedure as the Opta™.

You will need the latest

Arduino Mbed OS Portenta Boards

core version. If you do not have the latest version, please refer to the [Setting Up the Arduino IDE](#) section before proceeding.

You will need an Arduino sketch with memory partition operations and a certificate file. The required sketch is called **QSPIFormat** and can be found in the Arduino IDE by navigating to **File > Examples > STM32H747_System**. The necessary files are organized in the following directory structure:

```
1 QSPIFormat
2 └─ certificates.h
3 └─ QSPIFormat.ino
```

If you cannot find the **QSPIFormat** sketch in your board's examples, please make sure you have installed the **latest Arduino Mbed OS Portenta Boards core (version 4.3.1 or higher)**.



Compile and upload the following code to partition the memory of a Portenta Machine Control:

```

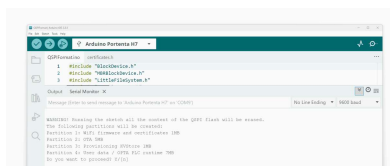
1  #include "BlockDev
2  #include "MBRBlockL
3  #include "LittleFil
4  #include "FATFileSy
5  #include "wiced_res
6  #include "certifica
7
8  #ifndef CORE_CM7
9      #error Format QSP
10 #endif
11
12 using namespace mbe
13
14 BlockDevice* root =
15 MBRBlockDevice wifi
16 MBRBlockDevice ota_
17 MBRBlockDevice kvst
18 MBRBlockDevice user
19 FATFileSystem wifi_
20 FATFileSystem ota_d
21 FileSystem * user_d
22
23 bool waitResponse()
24     bool confirmation
25     bool proceed = fa
26     while (confirmati
27         if (Serial.avai
28             char choice =

```

Once the memory partition code is uploaded, follow the instructions and wait for the process to finish. The upload target for the Portenta Machine Control is

Arduino Portenta H7, which will be automatically detected by the Arduino IDE.

The Arduino IDE's Serial Monitor should display memory partition process and results similar to those of the Opta™.



After this, you will be ready to use the full functionalities of the Portenta Machine Control.

Conclusion

In this tutorial, we showed how to partition the memory of an Opta™ and a Portenta Machine Control, enabling its full range of functionalities to be programmed using the Arduino ecosystem tools such as the Arduino IDE and the Arduino programming language.

We walked through initializing and erasing the QSPI Flash memory, partitioning and formatting the memory, and writing the Wi-Fi firmware and certificate data onto the device's memory.

Next Steps

As you move forward, you can explore the extensive Arduino ecosystem, leverage various libraries and hardware add-ons, and create powerful, connected, and secure industrial solutions using the Opta™. Check out our [Opta™ User Manual](#) to learn more about its hardware and software features.

To learn more about using the Portenta Machine Control within the Arduino IDE to its full capability, please check out the [Portenta Machine Control User](#)

Library Guide for using the

Arduino_PortentaMachineControl

library.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?